MIMIC

A Unified Platform for Astronomy Education, Research and Minihalo Detection

Technical Report
SE - 801 - Software Project Lab III

Supervised by:
Dr. B M Mainul Hossain
Professor
Institute of Information Technology
University of Dhaka

Submitted by:
Tasnim Mahfuz Nafis (BSSE 1327)
Institute of Information Technology
University of Dhaka

Submitted to:
SPL 3 Program Committee
Submission Date: 13 January 2026

Letter of Transmittal

13 January 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **MIMIC**: A Unified Platform for Astronomy Education, Research and Minihalo Detection.

Sir,
I am submitting the technical report for our Software Project Lab 3, "MIMIC: A Unified Platform for Astronomy Education, Research and Minihalo Detection".

This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I have tried to do my best, I welcome your feedback for improvement.

Thank you for your kind consideration.

Sincerely,
Tasnim Mahfuz Nafis
BSSE 1327
Institute of Information Technology
University of Dhaka

13.01.2026
—----------
Supervisor’s Signature

Acknowledgement

I am grateful towards my supervisor Dr. B M Mainul Hossain for allowing me to work on a topic that genuinely interests me. I am also grateful towards Dr. Khan Muhammad Bin Asad, Assistant Professor, Dept. of Physical Sciences, Independent University Bangladesh, for guiding me with the astronomical aspects of my project.

Abstract

MIMIC stands for Mining Minihalos in Clusters of Galaxies. This is a novel scientific Python package for detecting Cold Fronts and Minihalos. A Cold Front is a region of galaxy clusters that marks a sharp temperature change. A Minihalo is a distinct radio glow emitted by high-energy electrons found in the centers of some galaxy clusters. Professional astronomers can use this package with real Radio and X-ray telescope data. Our platform will also support agentic coding with the Mimic package. Agentic coding is a software development approach where AI agents autonomously plan and write codes in response to natural language queries. In this module, the user will receive step-by-step code generation, explanations and guidance. This feature will help new researchers learn how to use Mimic easily and effectively. Additionally, students can talk to an AI tutor on our platform to get help with their undergraduate astronomy courses. Overall, the platform is designed to serve both astronomy students and professional researchers in a single place.

Table of Contents

1. Introduction

MIMIC aims to be an useful platform for researchers, students and teachers alike - making it a space for research based astronomy education. The MIMIC scientific python package will be helpful for astronomers to probe research questions about the relationship between minihalos and cold fronts of galaxy clusters. To aid new researchers aiming to work with the mimic package, the unified platform will support agentic coding to provide step-by-step code generation and guidance. Students will be able to talk to an AI tutor fine-tuned on at least one undergrad level astronomy course materials. To support the students, registered teachers will be able to upload new course materials which will be incorporated with the AI chat assistant relying on the RAG (Retrieval-Augmented Generation) technology.

1.1 Motivation

Minihalos are very exotic astronomical objects. Very ancient collections of electrons with velocity comparable to that of light produce this distinct radio glow called Minihalo. So far, humankind has detected less than 30 Minihalos in the universe, making it even more exotic than black holes, and definitely less studied through scientific processes.

These days, data processing through efficient software engineering is a very integral part of astronomy research. One of the motivations for this project is to explore this intersection between software engineering and astronomy by making a tool for astronomers for testing the hypothesis - “Minihalos are formed inside cold fronts of galaxy clusters.”

To explore this hypothesis, we need both radio and x-ray images of galaxy clusters. Center for Astronomy, Space Science and Astrophysics (CASSA), Independent University of Bangladesh (IUB) has the radio image of some galaxy cluster centers for which publicly available raw x-ray data also exists - making this project feasible.

To explore the currently booming Artificial Intelligence, Fine-tuning and RAG (Retrieval-Augmented Generation) and develop a software3.0 project is the motivation for incorporating AI aided educational features aimed at students and teachers. Upon

completion, there is a chance of piloting the project in at least one of the astronomy minor courses of CASSA, IUB.

1.2 Scope

- Develop a scientific python package to test the relationship between minihalos and cold fronts of galaxy clusters.
- Provide students AI based chat support through fine-tuned foundation models.
- Allow teachers to upload new course materials and incorporate the information through RAG technology.
- Generate codes for the newly developed MIMIC package.

1.3 Purpose of Document

- Identify and analyze the requirements.
- Reduce the development effort by creating a well-structured plan.
- Design Use-Case Diagram.
- Design Data-based Modeling.
- Design Class-based Modeling.
- Defining the Archetype.
- Mapping requirements to the software architecture.
- Develop a preliminary test plan.

2. Project Requirements (QFD)

2.1 Normal Requirements:

- Registration and Authentication for students and teachers, with separate levels of authorization.
- Development of MIMIC package as a prototype.
- Storing user information and MIMIC package usage information in a database.
- Generating temperature maps, brightness maps, density maps, pressure maps and detecting cold fronts.

2.2 Expected Requirements:

- Making MIMIC work for both x-ray and radio data, leveraging existing CIAO and ds9 software components.
- Fine-tuning a foundation large language model with at least 3 credits worth of material in an undergraduate astronomy course.
- Allowing teachers the option to upload new materials to feed the underlying foundation model, using RAG technology.

2.3 Exciting Requirements:

- Incorporating agentic coding workflow to teach the MIMIC package.
- Dockerising the solution and scaling it to a classroom of size 30.
- Analyzing the usage pattern of the MIMIC software from database information and using the data to train a machine learning model to provide insights about the users.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. The scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. Through simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

On the other hand, A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

Adhering to these definitions, the Use Case Diagram of MIMIC follows:

3.1 Use Case Diagram

3.1.1 MIMIC

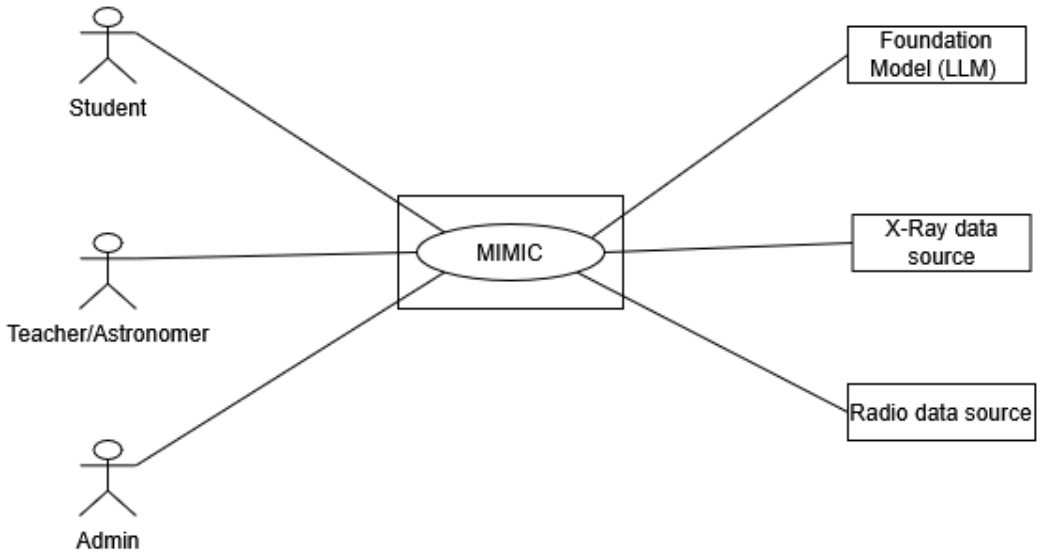

Fig 1: Use Case Level 0 (MIMIC)

Primary Actors: Student, Teacher/Astronomer, Admin
Secondary Actors: Foundation Model (LLM), X-Ray data source, Radio data source,
Goal in Context: The above diagram represents the high level overview of MIMIC.

3.1.2 Modules of MIMIC

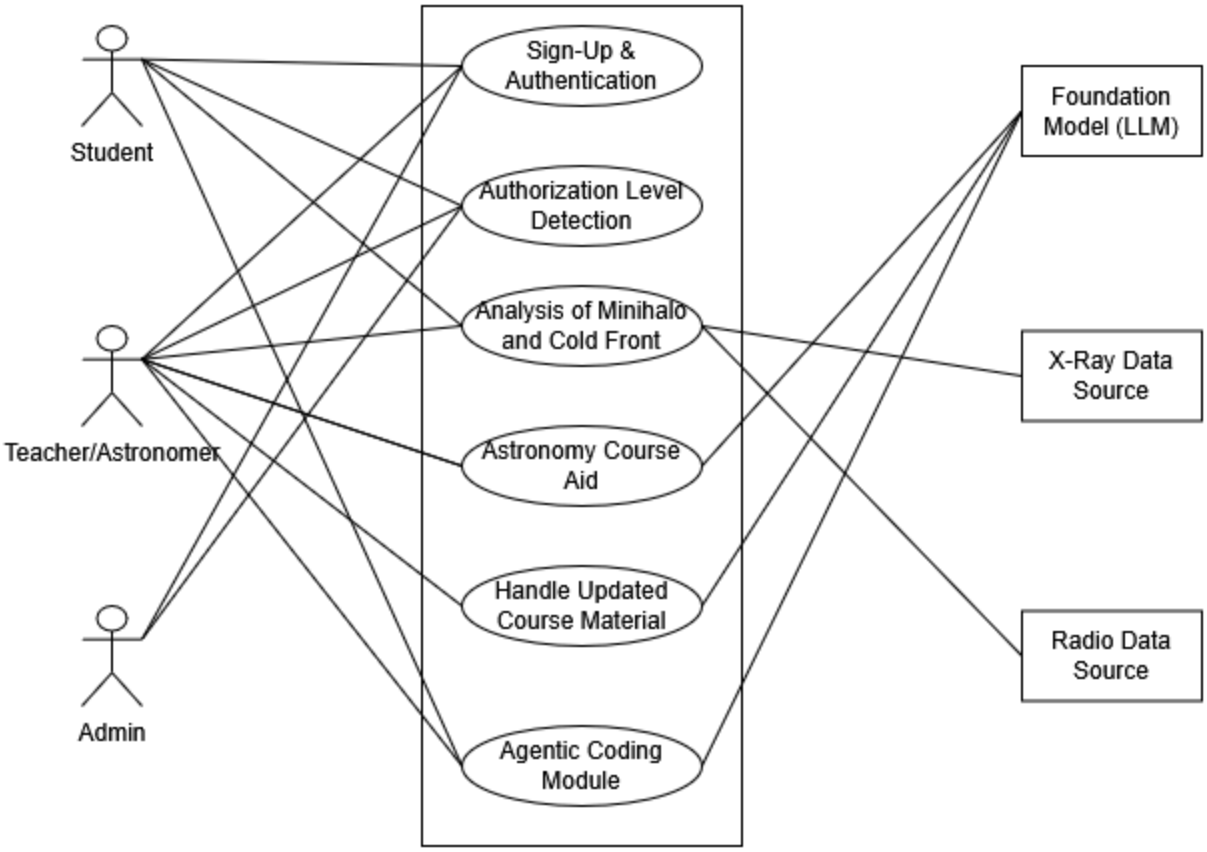

Fig 2: Use Case Level 1 (Modules of MIMIC)

Primary Actors: Student, Teacher/Astronomer, Admin
Secondary Actors: Foundation Model (LLM), X-Ray data source, Radio data source,
Goal in Context: The above diagram represents the main components of MIMIC.

3.1.3 Analysis of Minihalo and Cold Front

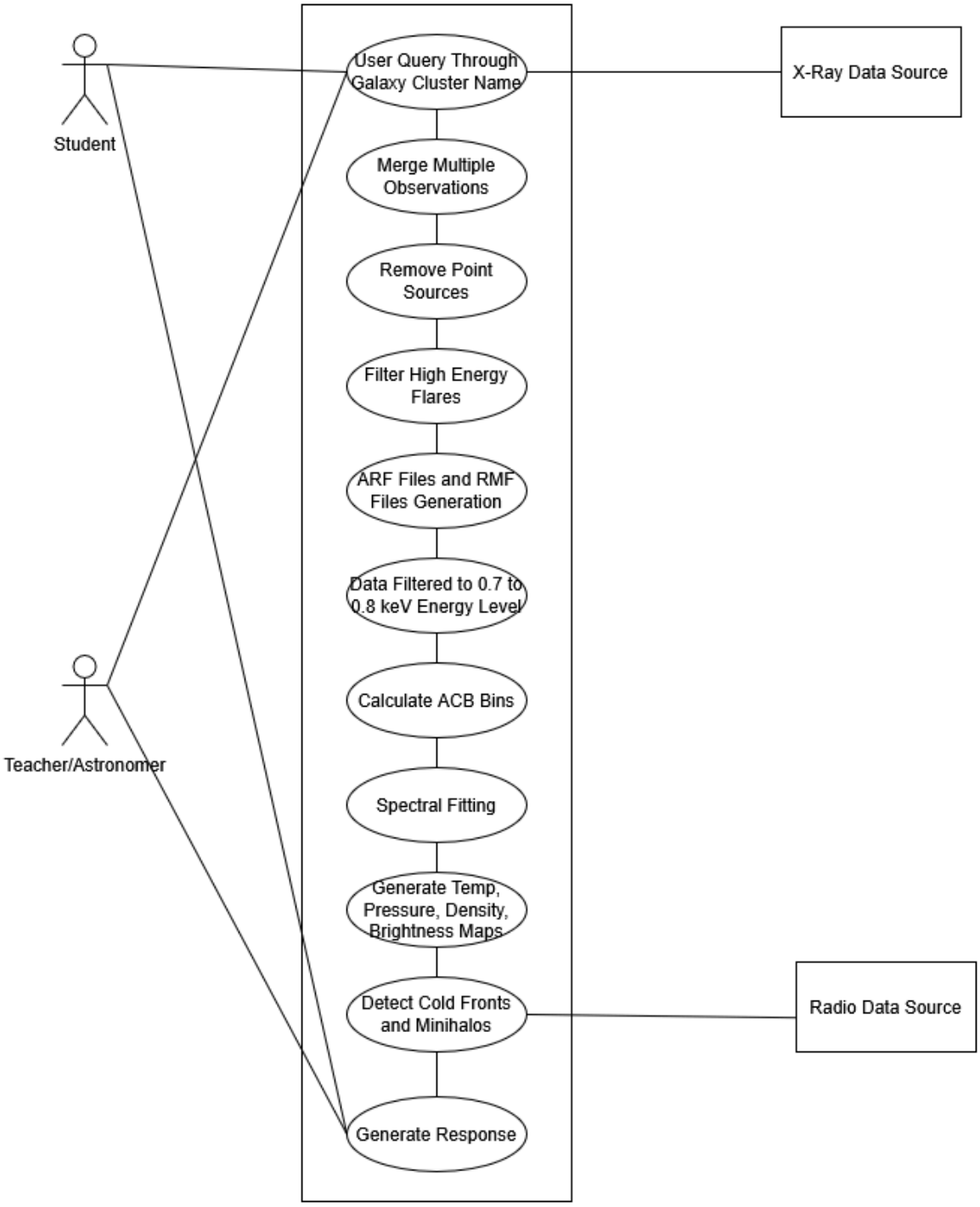

Fig 3: Use Case Level 2 (Analysis of Minihalo and Coldfront)

Primary Actors: Student, Teacher/Astronomer
Secondary Actors: , X-Ray data source, Radio data source,

Goal in Context: The above diagram represents the breakdown of the ‘Analysis of Cold Front and Minihalo’ component from Use Case Level 1.

4. Data Based Modeling

Data based modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The ER diagram highlighting the entities, attributes and their relations follows:

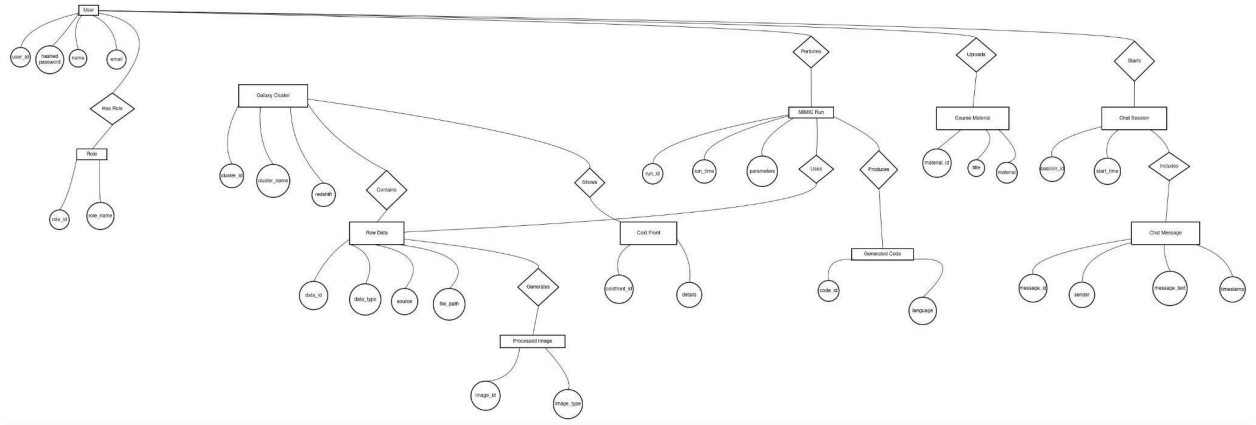

Fig 4: ER diagram of MIMIC

The database schema is depicted through the following tables, highlighting the primary and foreign keys, along with key constraints.

Table Name	Attribute Name	Data Type	Key/Constraints
User	**user_id**	INT	Primary Key
User	name	VARCHAR(100)	
User	email	VARCHAR(150)	Unique
User	hashed_password	VARCHAR(255)	

Table 1: User table for database schema

Table Name	Attribute Name	Data Type	Key/Constraints
Role	**role_id**	INT	Primary Key
Role	role_name	VARCHAR(50)	

Table 2: Role table for database schema

Table Name	Attribute Name	Data Type	Key/Constraints
UserRole	**user_id**	INT	Foreign Key
UserRole	**role_id**	INT	Foreign Key

Table 3: UserRole table for database schema

Table Name	Attribute Name	Data Type	Key/Constraints
GalaxyCluster	**cluster_id**	INT	Primary Key
GalaxyCluster	cluster_name	VARCHAR(100)	
GalaxyCluster	redshift	FLOAT	

Table 4: GalaxyCluster table for database schema

Table Name	Attribute Name	Data Type	Key/Constraints
RawData	**data_id**	INT	Primary Key
RawData	cluster_id	INT	Foreign Key
RawData	data_type	VARCHAR(50)	
RawData	source	VARCHAR(100)	
RawData	file_path	VARCHAR(255)	

Table 5: RawData table for database schema

Table Name	Attribute Name	Data Type	Key/Constraints
ProcessedImage	**image_id**	INT	Primary Key
ProcessedImage	data_id	INT	Foreign Key
ProcessedImage	image_type	VARCHAR(50)	

ProcessedImage	file_path	VARCHAR(255)	

Table 6: ProcessedImage table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
ColdFront	**coldfront_id**	INT	Primary Key
ColdFront	cluster_id	INT	Foreign Key
ColdFront	details	TEXT	

Table 7: ColdFront table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
MIMICRun	**run_id**	INT	Primary Key
MIMICRun	user_id	INT	Foreign Key
MIMICRun	data_id	INT	Foreign Key

MIMICRun	run_time	DATETIME	
MIMICRun	parameters	TEXT	

Table 8: MIMICRun table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
GeneratedCode	**code_id**	INT	Primary Key
GeneratedCode	run_id	INT	Foreign Key
GeneratedCode	language	VARCHAR(50)	
GeneratedCode	code_path	VARCHAR(255)	

Table 9: GeneratedCode table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
CourseMaterial	**material_id**	INT	Primary Key

CourseMaterial	user_id	INT	Foreign Key
CourseMaterial	title	VARCHAR(150)	
CourseMaterial	content_path	VARCHAR(255)	

Table 10: CourseMaterial table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
ChatSession	**session_id**	INT	Primary Key
ChatSession	user_id	INT	Foreign Key
ChatSession	start_time	DATETIME	

Table 11: ChatSession table for database schema

Table Name	**Attribute Name**	**Data Type**	**Key/Constraints**
ChatMessage	**message_id**	INT	Primary Key

ChatMessage	session_id	INT	Foreign Key
ChatMessage	sender	VARCHAR(20)	
ChatMessage	message_text	TEXT	
ChatMessage	timestamp	DATETIME	

Table 12: ChatMessage table for database schema

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

After detecting the nouns and verbs from the scenario, prospective classes and methods have been found, which is depicted through the following CRC diagram:

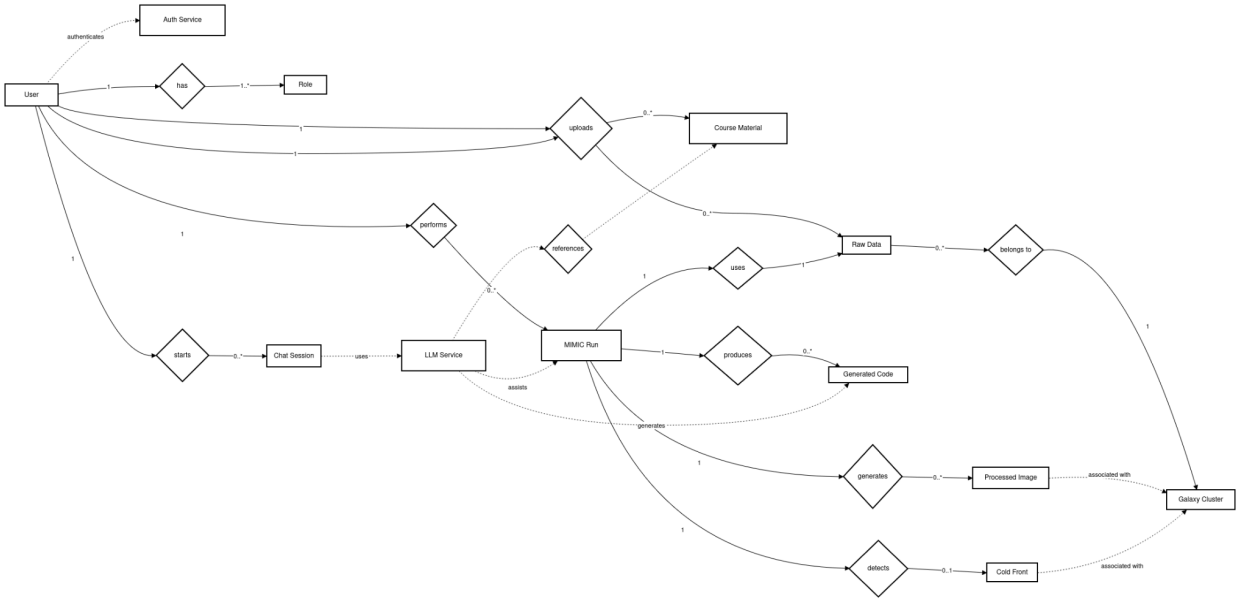

Fig 5: CRC (Class, Responsibility, Collaborator) diagram of MIMIC

The detailed Class Cards are depicted in the following tables:

Class: User	
Attributes	- userId, - name, - email, - hashedPassword
Methods	+ register(), + login(), + uploadData(), + startChat(), + runMIMIC()

Responsibilities	Authenticate and access the system; Upload raw observational data; Initiate scientific analysis; Interact with the AI chat system
Collaborators	AuthService, Role, RawData, MIMICRun, ChatSession, CourseMaterial

Table 13: CRC card for User class

Class: AuthService	
Attributes	- authMethod, - tokenExpiry
Methods	+ authenticateUser(credentials), + authorizeUser(role)
Responsibilities	Authenticate users; Enforce role-based authorization
Collaborators	User, Role

Table 14: CRC card for AuthService class

Class: Role	

Attributes	- roleId, - roleName
Methods	+ getPermissions()
Responsibilities	Define user access level; Support authorization decisions
Collaborators	User, AuthService

Table 15: CRC card for Role class

Class: RawData	
Attributes	- dataId, - dataType, - source, - filePath
Methods	+ storeData(), + retrieveData()

Responsibilities	Store X-ray and radio observational data; Maintain metadata for scientific analysis
Collaborators	User, GalaxyCluster, MIMICRun

Table 16: CRC card for RawData class

Class: GalaxyCluster	
Attributes	- clusterId, - clusterName, - redshift
Methods	+ linkData(), + getClusterInfo()
Responsibilities	Represent galaxy cluster metadata; Associate raw, processed, and cold-front data
Collaborators	RawData, ProcessedImage, ColdFront

Table 17: CRC card for GalaxyCluster class

Class: MIMICRun	
Attributes	- runId, - runTime, - parameters

Methods	+ executeAnalysis(), + generateOutputs()
Responsibilities	Execute the MIMIC scientific workflow; Generate maps and detect cold fronts; Track analysis execution details
Collaborators	User, RawData, ProcessedImage, ColdFront, GeneratedCode, LLMService

Table 18: CRC card for MIMICRun class

Class: ProcessedImage	
Attributes	- imageId, - imageType, - filePath
Methods	+ saveImage(), + loadImage()
Responsibilities	Store generated scientific maps; Support visualization and analysis
Collaborators	MIMICRun, GalaxyCluster

Table 19: CRC card for ProcessedImage class

Class: ColdFront	
Attributes	- coldfrontId, - details
Methods	+ storeDetection(), + getDetails()
Responsibilities	Store cold-front detection results; Maintain diagnostic information
Collaborators	MIMICRun, GalaxyCluster

Table 20: CRC card for ColdFront class

Class: GeneratedCode	
Attributes	- codeId, - language, - codePath
Methods	+ saveCode(), + retrieveCode()
Responsibilities	Store auto-generated analysis code; Support agentic coding and learning
Collaborators	MIMICRun, LLMService

Table 21: CRC card for GeneratedCode class

Class: CourseMaterial	
Attributes	- materialId, - title, - contentPath
Methods	+ uploadMaterial(), + retrieveMaterial()
Responsibilities	Store educational materials; Act as knowledge source for AI (RAG)
Collaborators	User, LLMService

Table 22: CRC card for CourseMaterial class

Class: ChatSession	
Attributes	- sessionId, - startTime
Methods	+ startSession(), + logMessage()
Responsibilities	Manage AI–user interactions; Maintain chat history

Collaborators	User, LLMService

Table 23: CRC card for ChatSession class

Class: LLMService	
Attributes	- modelName, - version, - apiEndpoint, - parameters, - contextData
Methods	+ assistUser(query), + generateCode(context), + summarizeData(data), + answerQuestion(content)
Responsibilities	Provide AI-based chat assistance; Assist MIMIC analysis and code generation; Use course materials via RAG; Summarize scientific results
Collaborators	ChatSession, CourseMaterial, MIMICRun, GeneratedCode

Table 24: CRC card for LLMService class

A more detailed CRC diagram is provided in the following page containing attributes, methods, responsibilities and collaborators explicitly mentioned.

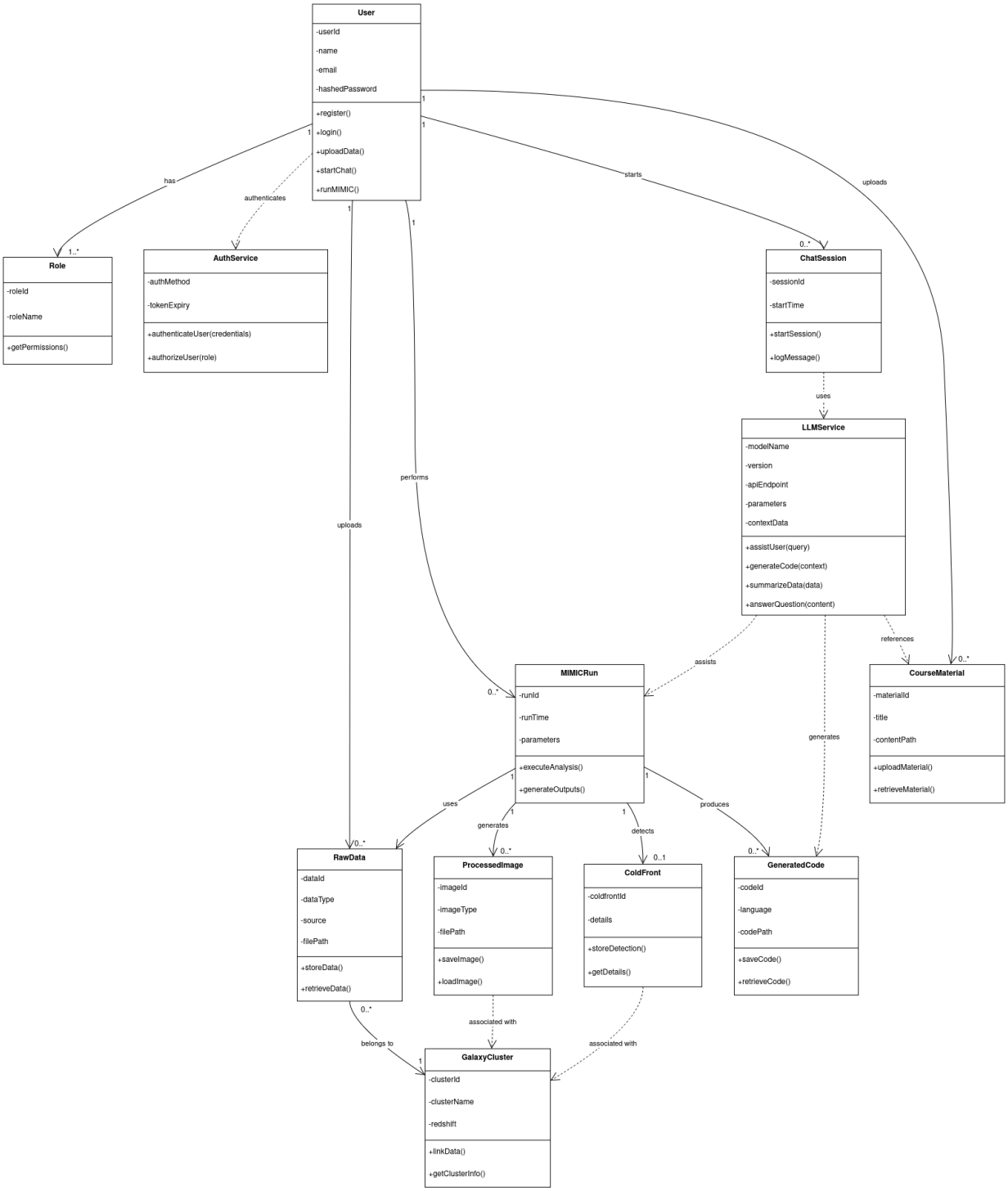

Fig 6: CRC diagram with attributes and methods

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

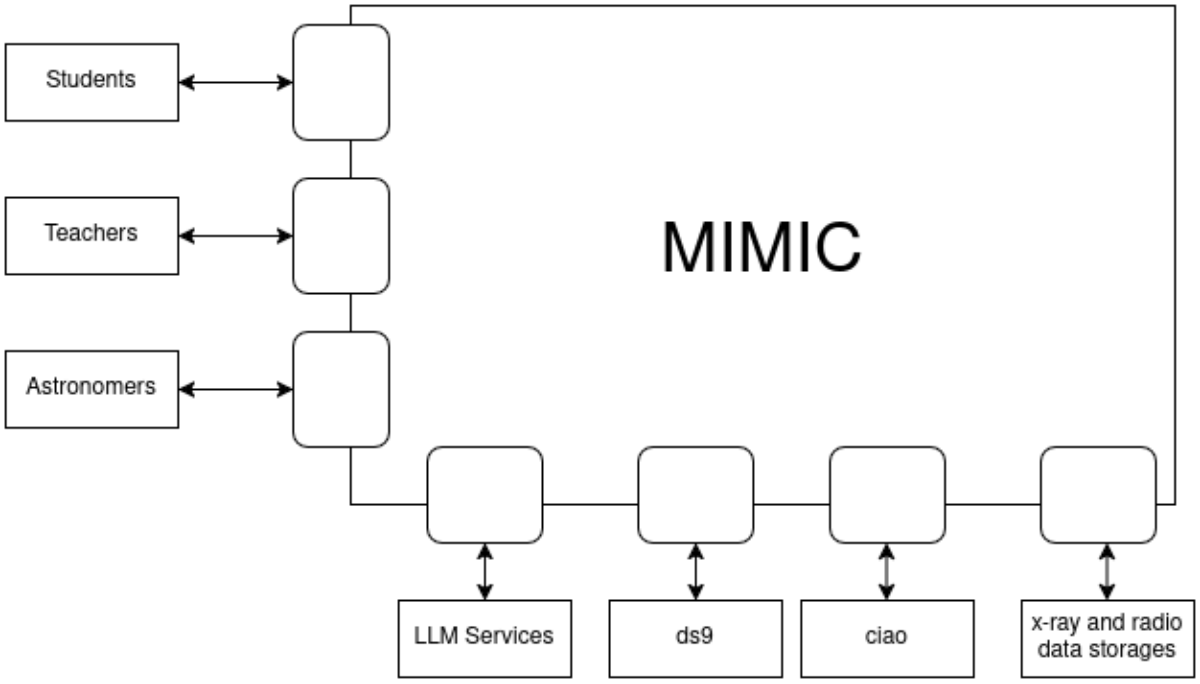

FIg 7: Architectural context diagram of MIMIC

In the above diagram-
Users - Students, Teachers and Astronomers.
Subordinate Systems - LLM Services, ds9, ciao, x-ray and radio data storages.

6.2 Archetypes

The archetypes for the tool are defined below-

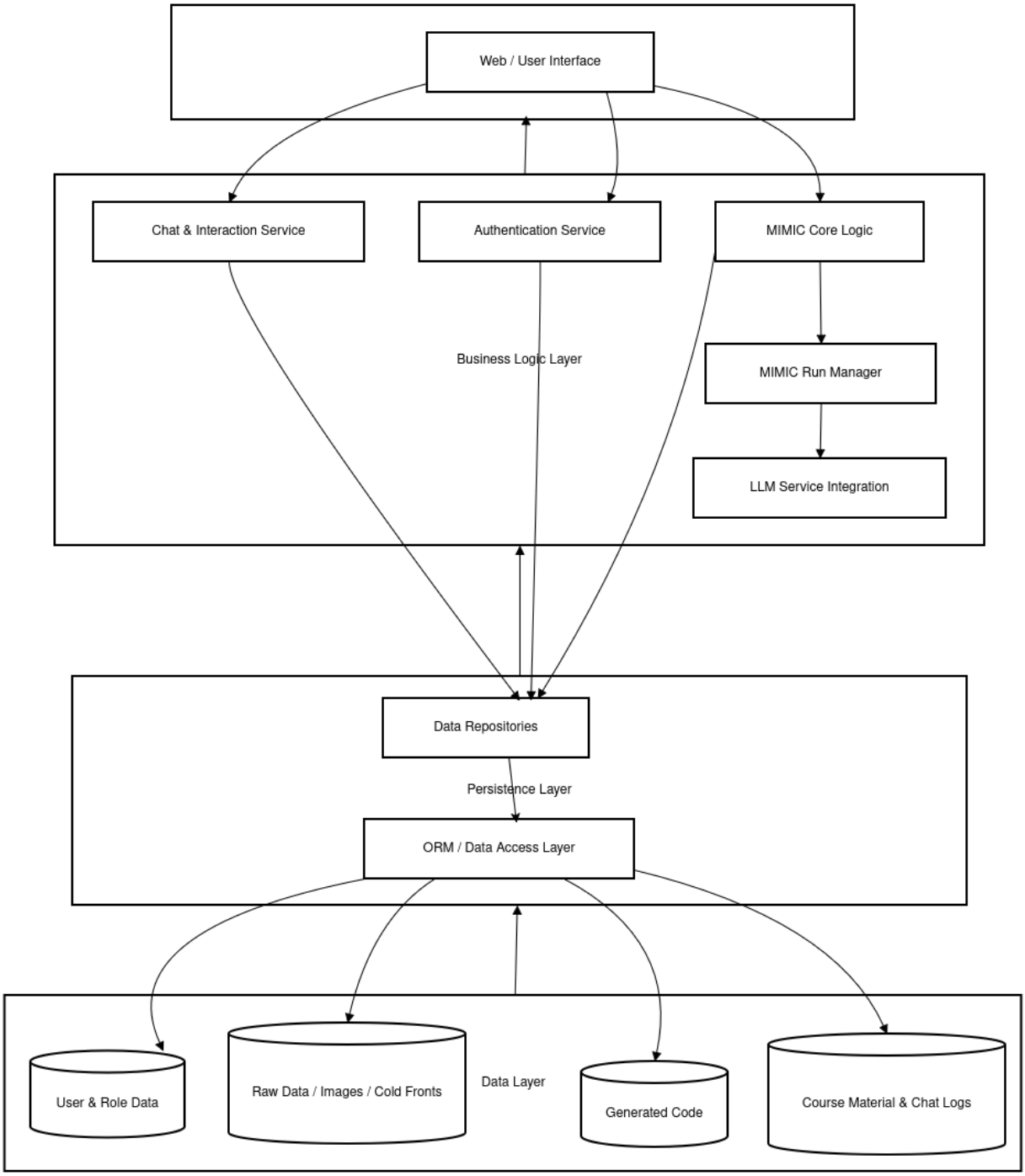

Fig 8: Archetype of MIMIC

The archetype diagram, containing presentation, business logic, persistent and data layer in vertical modular order, exemplifies “Dependency Inversion Principle”, which is often summarized by this definitive line:

> *"Abstractions should not depend on details; details should depend on abstractions."*

6.3 Deployment Diagram

After developing the MIMIC scientific package, deployment will follow the structure below:

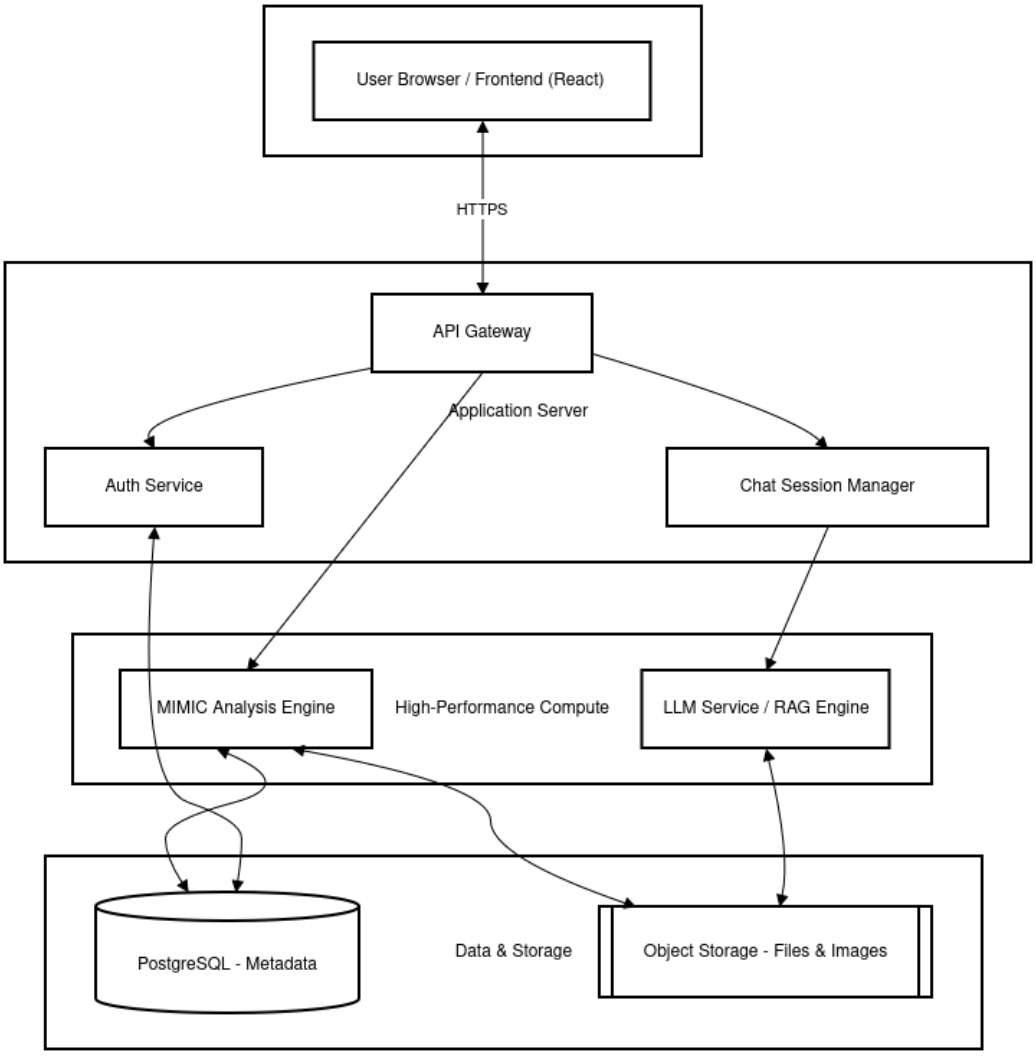

Fig 9: Deployment Plan Diagram of MIMIC

The backend will rely on python-based technologies, including fastapi. As for database, postgreSQL will be used. The frontend will be developed with React.

7. Preliminary Test Plan

The following table highlights the basic high level testing plans-

#	Title	Scenario	Steps	Expected Outcome
1	**User Authentication**	Verify secure login flow.	1. Enter valid email/password. 2. Submit form.	JWT token is issued; user redirected to dashboard.
2	**Raw Data Ingestion**	Upload a valid X-ray FITS file.	1. Select .fits file. 2. Assign to Galaxy Cluster. 3. Click Upload.	File saved to file_path; metadata entry created in RawData table.

3	**Cluster Linking**	Associate data with specific clusters.	1. Upload data. 2. Select "Perseus Cluster" from dropdown.	RawData.cluster_id correctly maps to GalaxyCluster.cluster_id.
4	**MIMIC Execution**	Run a standard analysis.	1. Select RawData ID. 2. Input parameters. 3. Trigger executeAnalysis().	MIMICRun record created; ProcessedImage generated and saved to path.
5	**Cold Front Precision**	Detect known cold front features.	1. Run MIMIC on data with high-pressure gradients.	ColdFront table populated with specific coordinates and detailed text.
6	**RAG Knowledge Retrieval**	LLM uses course materials for context.	1. Ask chat: "Define our lab's X-ray cleaning steps."	LLM cites the specific PDF from the CourseMaterial table.

7	**Code Generation Logic**	Agentic coding from analysis.	1. Request Python script for specific plot.	GeneratedCode record created; valid .py file stored in code_path.
8	**Session Persistence**	Verify chat history tracking.	1. Start chatting. 2. Send 3 messages. 3. Refresh page.	ChatSession and ChatMessage tables retrieve and display the 3 messages.
9	**Role-Based Access**	Restrict "Guest" from running MIMIC.	1. Log in as Guest. 2. Attempt to trigger runMIMIC().	Access Denied error; no entry created in MIMICRun table.
10	**Data Integrity**	Cascade delete check.	1. Delete a GalaxyCluster record.	All associated RawData and ColdFront records are handled (deleted or orphaned) per FK rules.

#	Title	Scenario	Steps	Expected Outcome
11	**LLM Hallucination Check**	Prevent AI from "guessing" data.	1. Ask about a cluster not in the database.	LLM states it does not have data for that cluster rather than making up stats.
12	**Analysis Summary Quality**	LLM summarizes MIMIC results.	1. Complete a MIMIC run. 2. Click "Summarize Results."	LLM provides a concise text summary of the ColdFront.details and run_time.

#	Title	Scenario	Steps	Expected Outcome
13	**Concurrent Processing**	Multiple users running MIMIC.	1. 10 users trigger runMIMIC() simultaneously.	Server manages queue; all 10 runs complete without database deadlock.
14	**Large File Handling**	Uploading 2GB+ observation file.	1. Upload a maximum-size FITS file.	The system handles the stream without memory overflow; file_path is correctly logged.
15	**Rapid Chat Input**	Flooding the LLM service.	1. Send 50 messages in 10 seconds.	API rate limiting triggers or sessions remains stable without crashing.

Table 25: High level testing plan for MIMIC

8. Conclusion

In this report, I have noted the technical details of MIMIC, including Use Case diagrams, CRC diagrams, Architectural diagrams, Deployment plan and high level testing plans. I have also clearly defined the scope of the project. Some parts of the project will be developed for prototyping and exploring the aspects of Software Engineering 3.0, whereas some parts will be developed rigorously. The extent of expected development for every separate module by the project completion is explicitly defined. By the end of the project, MIMIC aims to be useful for astronomy research and education, by blending cutting edge software engineering and astronomy together. It is to be noted that during the preparation of the document, help were taken from LLMs like chatGPT and Gemini (basic versions) in order to automate some repetitive aspects. The design and work, however, is purely original.

9. Motivating paper

K S Trehaeven, V Parekh, N Oozeer, B Hugo, O Smirnov, G Bernardi, K Knowles, C Tasse, K M B Asad, S Giacintucci, Mining mini-halos with MeerKAT I. Calibration and imaging, Monthly Notices of the Royal Astronomical Society, Volume 520, Issue 3, April 2023, Pages 4410–4426, https://doi.org/10.1093/mnras/stad391